Labo 09 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Scripts contre Compose

Réponse. (1) **L'état** : `up` compare le fichier aux conteneurs existants et ne recrée que ce qui a changé ; un script relance tout ou échoue sur « name already in use ». (2) **L'ordre et l'attente** : `depends_on` + `healthcheck` font attendre qu'un service soit prêt ; un script enchaîne des `sleep`. (3) **Le périmètre** : `down` connaît exactement ce que le projet a créé (conteneurs, réseau, pod) et le retire sans toucher au reste ; un script de nettoyage oublie toujours quelque chose, ou supprime trop. S'y ajoutent la fusion d'environnements (`-f`, override) et la lisibilité par un nouvel arrivant.

Pourquoi. Compose est **déclaratif** : il décrit un état cible et calcule la différence. Un script est **impératif** : il décrit des actions, sans connaître le résultat attendu.

Nuance. Un script reste utile pour ce que Compose ne couvre pas (créer un secret du moteur, initialiser un fichier). Les deux cohabitent : le script appelle Compose.

Exemple.

```
podman compose up -d && podman compose up -d     # la seconde fois : rien à faire, aucun conteneur recréé
```

Question 2 — `.env` n'entre pas dans le conteneur

Réponse. `.env` est lu par **Compose**, pour substituer les `${…}` du fichier YAML (`${API_TAG:-1.0}`) ; il n'est transmis à aucun conteneur. `APP_MESSAGE` devait être dans `environment:` du service, ou dans un fichier référencé par `env_file:` (`api.env`), qui, lui, est passé au conteneur comme `--env-file`.

Pourquoi. Deux mécanismes, deux moments : la substitution se fait à la lecture du YAML, l'environnement se construit à la création du conteneur.

Nuance. Une variable du **shell** est aussi utilisée pour la substitution, avant `.env`. Et `podman compose config` montre le YAML substitué : si `${APP_MESSAGE}` n'y apparaît pas, `.env` n'a aucun effet.

Exemple.

```
podman compose config | grep -c 'bonjour depuis dotenv'     # 0 : la variable n'est utilisée nulle part
podman compose exec api env | grep APP_MESSAGE              # la valeur de api.env
```

Question 3 — `depends_on` court et long

Réponse. Forme courte : `db` est **créé et démarré** avant `api`, mais `api` part immédiatement ; PostgreSQL n'accepte pas encore de connexion, Spring Boot échoue (`Connection refused`, `HikariPool … Exception during pool initialization`) et le conteneur s'arrête avec un code d'erreur — sans `restart`, il reste `Exited`. Forme longue : `api` n'est démarré qu'une fois le

healthcheck de `db` passé (`healthy`) ; Spring Boot trouve la base, tout est `Up`. La condition exige un `healthcheck` : sans lui, `db` n'a pas d'état de santé, et podman-compose refuse ou attend indéfiniment.

Pourquoi. « Démarré » est un état du conteneur ; « prêt » est un état de l'application, que seul un healthcheck peut observer.

Nuance. Avec `restart: unless-stopped`, la forme courte « finit par marcher » après quelques redémarrages : c'est fragile et cela cache le problème. Notre fausse API ne se connecte pas à la base ; l'effet se voit sur l'**ordre** (`podman compose ps` montre `api` démarré cinq secondes après `db`), pas sur un échec.

Exemple.

```
podman compose ps     # db  Up 11 seconds (healthy)  /  api  Up 6 seconds (healthy)  /  web  Up 1 second
```

Question 4 — `curl` absent de l'image

Réponse. `eclipse-temurin:21-jre-alpine` ne contient pas `curl` : chaque exécution du healthcheck échoue (`exec: "curl": executable file not found`), et après `retries` échecs l'état devient `unhealthy` — `web`, qui attend `service_healthy`, ne démarre jamais. Diagnostic : `podman inspect stack_api_1 --format '{{json .State.Health.Log}}'`, ou `podman healthcheck run stack_api_1`. Correction : utiliser `wget` (BusyBox, présent dans Alpine) : `["CMD", "wget", "-qO-", "http://localhost:8080/actuator/health"]`.

Pourquoi. Le healthcheck s'exécute **dans** le conteneur, avec les binaires de l'image : une commande absente est un échec comme un autre.

Nuance. Les images distroless n'ont ni `curl` ni `wget` ni shell : le healthcheck doit alors être un binaire fourni (ou, en Kubernetes, une sonde HTTP exécutée par le kubelet, sans commande).

Exemple.

```
podman compose exec api wget -qO- http://localhost:8080/actuator/health     # {"status":"UP"} : la commande existe
podman compose exec api curl                                                # executable file not found
```

Question 5 — Volume nommé ou bind mount pour la base

Réponse. `pgdata:` : Compose crée le volume `stack_pgdata`, Podman en gère le répertoire, les fichiers appartiennent à l'UID traduit de `postgres` (`100069` sur WSL), `down` le conserve, `down -v` le supprime. `./data` : vous créez le dossier (ou Podman le crée, appartenant à vous), les fichiers y ont le même UID traduit, donc `ls ./data` répond `Permission denied` et un `git status` tombe dessus ; `down -v` **ne le touche pas**. Pour la base : le volume nommé — sauvegardable par `podman volume export` (labo 06), hors du dépôt, et sans surprises de permissions.

Pourquoi. Compose ne gère que ce qu'il déclare ; un chemin hôte est à vous, un volume est à lui.

Nuance. Le bind mount est le bon choix pour la **configuration** (`nginx-proxy.conf`, `index.html`) : versionnée, lisible, modifiable à chaud.

Exemple.

```
podman compose down && podman volume ls | grep stack_pgdata     # toujours là
podman compose down -v && podman volume ls | grep -c stack_pgdata   # 0
```

Question 6 — Le projet fantôme

Réponse. Sans `name:`, le projet s'appelle comme le dossier : `stack` d'un côté, `stack-v2` de l'autre. Le second `up` a créé `stack-v2_default`, `stack-v2_pgdata` (vide) et `stack-v2_db_1` : une seconde stack, qui ne voit pas le volume de la première. Rien n'est perdu : `podman volume ls` montre les deux volumes. Deux parades : `name: stack` dans le fichier, ou `podman compose -p stack up -d` (ou `COMPOSE_PROJECT_NAME`).

Pourquoi. Le nom du projet est le préfixe de toutes les ressources ; changer de nom, c'est changer de monde.

Nuance. Deux projets côte à côte sont parfois **voulus** (deux branches, deux versions) — à condition de ne pas publier les mêmes ports.

Exemple.

```
podman volume ls --format '{{.Name}}'     # stack_pgdata  stack-v2_pgdata
podman compose -p stack-v2 down -v         # retirer le fantôme, garder l'original
```

Question 7 — Deux réponses pour un conteneur

Réponse. `podman exec` dit la vérité : il affiche l'environnement **du processus**, `prod`. `podman compose exec` relit les fichiers Compose **par défaut** (`compose.yaml` + `compose.override.yaml`, puisque aucun `-f` n'est donné), en déduit l'environnement du service et l'injecte dans la commande `exec` : `dev` vient de l'override, pas du conteneur. Pour éviter : donner à `exec` les mêmes `-f` qu'à `up`, fixer `COMPOSE_FILE=compose.yaml:compose.prod.yaml` dans le shell, ou utiliser `podman exec`.

Pourquoi. podman-compose passe `--env` à `podman exec` pour reproduire l'environnement déclaré ; il ne sait pas avec quels fichiers le conteneur a été créé.

Nuance. Docker Compose a le même comportement pour le choix des fichiers : `-f` n'est jamais mémorisé. `COMPOSE_FILE` est la convention des deux outils.

Exemple.

```
podman compose -f compose.yaml -f compose.prod.yaml exec api env | grep APP_PROFILE   # prod
```

Question 8 — `restart: unless-stopped` sans daemon

Réponse. Avec Docker, le daemon démarre avec la machine et relance les conteneurs dont la politique l'exige : la stack revient seule. Avec Podman rootless, il n'y a **pas de daemon** : après `wsl --shutdown`, personne ne relit les politiques de redémarrage ; les conteneurs restent `Exited` jusqu'à un `podman compose up -d` ou `podman start`. La politique n'agit que **tant que Podman tourne** (crash de l'application → relance). Pour un démarrage automatique : des unités **systemd** générées par Quadlet (labo 10), ou `podman compose up -d` dans une unité `systemd --user` avec *linger*.

Pourquoi. `--restart` est exécuté par `conmon` et Podman pour un conteneur vivant ; il n'existe aucun processus permanent pour agir au démarrage du système.

Nuance. `podman generate systemd` (déprécié) et Quadlet produisent des unités qui, elles, sont relancées par systemd au boot — c'est la réponse Podman à `restart: always`.

Exemple.

```
podman inspect stack_api_1 --format '{{.HostConfig.RestartPolicy.Name}}'   # unless-stopped
wsl --shutdown   # (depuis Windows) puis : podman ps -a → Exited
```

Question 9 — Deux API derrière nginx

Réponse. Les deux conteneurs portent l'**alias** `api` sur le réseau `stack_default` ; le DNS interne renvoie les deux adresses, et nginx (qui résout au démarrage) prend celle(s) qu'il obtient — en pratique une répartition approximative, pas un vrai équilibrage. `--scale web=2` échoue parce que les deux instances voudraient publier `18080` sur l'hôte : conflit de port. Avec podman-compose, `down` ne connaît que l'instance 1 : il faut retirer `stack_api_2` nommément (`podman rm -f -t 0 stack_api_2`) avant `down`, sinon `db` ne peut pas être supprimé (dépendance).

Pourquoi. Le `scale` est une multiplication de conteneurs sous un même nom de service ; la répartition réelle est l'affaire d'un équilibreur (Kubernetes Service, labo 13).

Nuance. Docker Compose gère le scale à la baisse et au `down`. C'est une des limites concrètes de podman-compose à connaître.

Exemple.

```
podman run --rm --network stack_default alpine nslookup api 10.89.0.1 | grep Address   # deux adresses
podman rm -f -t 0 stack_api_2 && podman compose down -v
```

Question 10 — « name already in use »

Réponse. Un `stack_db_1` existe déjà, laissé par un `down` incomplet (par exemple après un `--scale`, question 9) ou par un `podman run --name` manuel. `up` crée un conteneur neuf d'après le fichier et ne peut pas « adopter » un conteneur qui ne porte pas ses étiquettes de projet ou dont la configuration diffère. Réparation : `podman ps -a` pour voir, `podman rm -f -t 0 stack_db_1` (le **volume** `stack_pgdata` n'est pas touché), puis `up -d`.

Pourquoi. Les noms de conteneurs sont uniques par moteur ; Compose ne supprime jamais silencieusement un conteneur qu'il ne reconnaît pas.

Nuance. `podman compose down` d'abord est la bonne habitude ; si lui-même échoue, le nettoyage nommé est la seule voie — jamais `podman rm -af` sur une machine partagée.

Exemple.

```
podman ps -a --filter name=stack_ --format '{{.Names}} {{.Status}}'
podman rm -f -t 0 stack_db_1 && podman compose up -d && podman volume ls | grep stack_pgdata
```

Question 11 — Secret par fichier

Réponse. Avec `secrets:` + `_FILE` : `podman compose config` montre `file: ./db_password.txt` (le chemin), `podman inspect` montre `POSTGRES_PASSWORD_FILE=/run/secrets/db_password`, et Git ne voit que le fichier YAML — **si** `db_password.txt` est dans `.gitignore`. Avec `environment: POSTGRES_PASSWORD: S3cr3t` : la valeur est dans le YAML (donc dans Git), dans `config`, dans `inspect`. `db_password.txt` : hors Git, droits `600`, un par environnement, ou remplacé par un secret du moteur avec `external: true` (`podman secret create` au préalable, labo 08).

Pourquoi. Compose monte le fichier dans `/run/secrets/<nom>` ; la valeur ne transite par aucune métadonnée.

Nuance. Docker Compose hors Swarm fait exactement la même chose (un montage de fichier) ; `external: true` y désigne un secret Swarm, chez Podman un `podman secret`.

Exemple.

```
podman compose config | grep -A1 'db_password:'          # file: ./db_password.txt
podman inspect stack_db_1 --format '{{json .Config.Env}}' | grep -c S3cr3t   # 0
```

Question 12 — Correspondances Kubernetes

Réponse. Service avec `image` → **Deployment** (et son Pod) ; `ports` → **Service** (ClusterIP/NodePort) et, pour l'extérieur, **Ingress** ; volume `pgdata` → **PersistentVolumeClaim** (et StatefulSet pour une base) ; `healthcheck` → **readiness/liveness probes** ; `depends_on` → **pas d'équivalent direct** : Kubernetes ne gère pas l'ordre de démarrage, chaque Pod réessaie jusqu'à ce que ses dépendances répondent, et la *readiness probe* évite d'envoyer du trafic avant ; réseau implicite → le réseau du **cluster** et le DNS de CoreDNS ; `env_file` → **ConfigMap** ; `secrets` → **Secret**.

Pourquoi. Kubernetes est conçu pour des composants qui tombent et reviennent : il remplace « attendre » par « réessayer et ne pas router tant que ce n'est pas prêt ».

Nuance. Des *init containers* peuvent attendre une dépendance, mais c'est un contournement, pas le modèle.

Exemple.

```
podman compose config > stack.yaml   # point de départ pour kompose, ou pour écrire les
manifestes du labo 12
```

Question 13 — Compose en production

Réponse. Raisonnable pour une application interne à faible trafic, sans exigence de disponibilité, sur une VM gérée par une petite équipe — ce qui décrit beaucoup d'applications réelles. Ce que Compose ne fournit pas : (1) la **haute disponibilité** (une VM, un point de panne) ; (2) la **mise à jour sans interruption** (un `up` recrée le conteneur, le service tombe quelques secondes) ; (3) la **répartition sur plusieurs machines** et l'autoscaling ; (4) le **redémarrage automatique** en rootless sans daemon, et la gestion chiffrée des secrets. Le signal : dès qu'on écrit des scripts pour faire ce que Compose ne fait pas (bascule, réplication, rolling update), il est temps de passer à un orchestrateur.

Pourquoi. Compose est un outil de **composition locale**, pas un ordonnanceur de cluster.

Nuance. Entre les deux existent des solutions simples : Quadlet + systemd pour le redémarrage, un équilibreur devant deux VM Compose, ou k3s mono-nœud (labo 11), qui apporte les mécanismes Kubernetes sans la taille d'un cluster.

Exemple.

```
podman compose -f compose.yaml -f compose.prod.yaml up -d && podman compose ps   # une VM, trois conteneurs, un point de panne
```